CS 4365 Artificial Intelligence

Final Project

Due: Sunday, April 26, 2026

Instructions:

1. The final project builds upon the programming component introduced in assignment 1.
2. Your solution to this project must be submitted via eLearning.
3. The free late days do not apply to the final project.
4. **You may work individually or in a group of two**. Only one submission should be made per group. If you work in a group, **make sure to indicate both group members when submitting through eLearning**.

Task Description

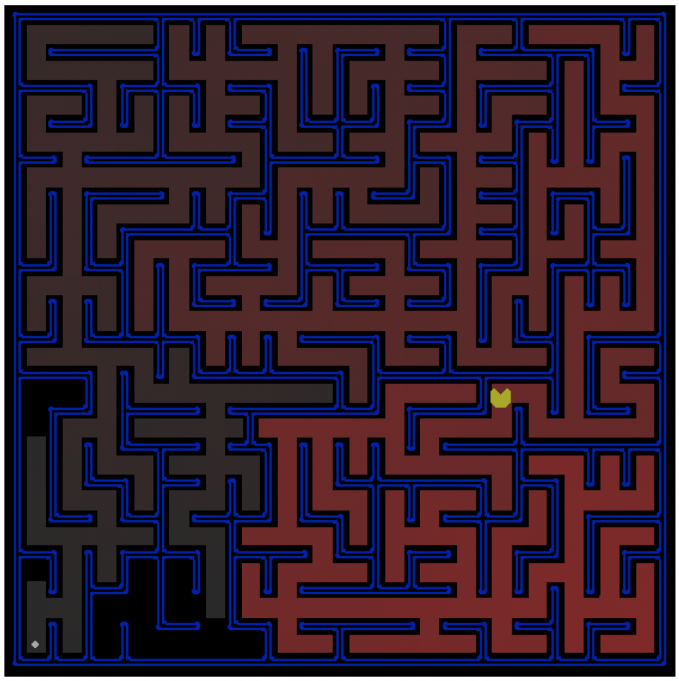

Figure 1: All those colored walls, Mazes give Pacman the blues, So teach him to search.

0.1 Introduction

In this project, your Pacman agent will find paths through his maze world, both to reach a particular location and to collect food efficiently. You will build general search algorithms and apply them to Pacman scenarios.

This project includes an autograder for you to grade your answers on your machine. This can be run with the command:

```
python autograder.py
```

Note the autograder will also grade Question q1 to Question q6, but we only consider Question q7 to Question q8 in this project.

You can also select individual questions for the autograder to run. For example, if you wanted to run question 7 on a project, you would run the command:

```
python autograder.py -q q7
```

The code for this project consists of several Python files, some of which you will need to read and understand in order to complete the assignment, and some of which you can ignore. You can download all the code and supporting files as a zip from eLearning.

0.2 Files you'll edit:

- **search.py**

  - Where all of your search algorithms will reside.

- **searchAgents.py**
  - Where all of your search-based agents will reside.

0.3 Files you might want to look at:

- **pacman.py**
  - The main file that runs Pacman games. This file describes a Pacman GameState type, which you use in this project.
- **game.py**
  - The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.
- **util.py**
  - Useful data structures for implementing search algorithms.

0.4 File Download:

Please see the eLearning.

0.5 Files to Submit:

You will fill in portions of **search.py** and **searchAgents.py** during the assignment. You should submit these files with your code and comments. Please *do not* change the other files in this distribution or submit any of our original files other than these files.

0.6 Evaluation:

Your code will be autograded for technical correctness. Please *do not* change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder. However, the correctness of your implementation – not the autograder's judgements – will be the final judge of your score. If necessary, we will review and grade assignments individually to ensure that you receive due credit for your work.

0.7 Academic Dishonesty

We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. These cheat detectors are quite hard to fool, *so please don't try.* We trust you all to submit your own work only; *please* don't let us down. If you do, we will pursue the strongest consequences available to us.

0.8 Getting Help

You are not alone! If you find yourself stuck on something, contact the course staff for help. Office hours and eLearning are there for your support; please use them. If you can't make our office hours, talk to our TA. We want these projects to be rewarding and instructional, not frustrating and demoralizing. But, we don't know when or how to help unless you ask.

Welcome to Pacman (Again)

After downloading the code (**final_project.zip**), unzipping it, and changing to the directory, you should be able to play a game of Pacman by typing the following at the command line:

```
python pacman.py
```

Note: The exact command you should be using depends on your distribution and python installation.

Pacman lives in a shiny blue world of twisting corridors and tasty round treats. Navigating this world efficiently will be Pacman's first step in mastering his domain.

The simplest agent in **searchAgents.py** is called the **GoWestAgent**, which always goes West (a trivial reflex agent). This agent can occasionally win:

```
python pacman.py --layout testMaze --pacman GoWestAgent
```

But, things get ugly for this agent when turning is required:

```
python pacman.py --layout tinyMaze --pacman GoWestAgent
```

If Pacman gets stuck, you can exit the game by typing CTRL-c into your terminal.

Soon, your agent will solve not only **tinyMaze**, but any maze you want.

Note that **pacman.py** supports a number of options that can each be expressed in a long way (e.g., **--layout**) or a short way (e.g., **-l**). You can see the list of all options and their default values via:

```
python pacman.py -h
```

Question 7 (50 points): Eating All The Dots

Note: this question builds upon question 4 in the assignment 1 and the solution to question 4 has been provided in the code.

We'll solve a hard search problem: eating all the Pacman food in as few steps as possible. For this, we'll need a new search problem definition which formalizes the food-clearing problem:

FoodSearchProblem in **searchAgents.py** (implemented for you). A solution is defined to be a path that collects all of the food in the Pacman world. For the present project, solutions do not take into account any ghosts or power pellets; solutions only depend on the placement of walls, regular food and Pacman. With the provided code, **A*** with a null heuristic (equivalent to uniform-cost search) should quickly find an optimal solution to **testSearch** with no code change on your part (total cost of 7).

```
python pacman.py -l testSearch -p AStarFoodSearchAgent
```

Note: **AStarFoodSearchAgent** is a shortcut for

```
-p SearchAgent -a fn=astar,prob=FoodSearchProblem,
heuristic=foodHeuristic
```

You should find that UCS starts to slow down even for the seemingly simple **tinySearch**. As a reference, our implementation takes 2.5 seconds to find a path of length 27 after expanding 5057 search nodes.

Fill in **foodHeuristic** in **searchAgents.py** with a **consistent** heuristic for the **FoodSearchProblem**. Try your agent on the **trickySearch** board:

```
python pacman.py -l trickySearch -p AStarFoodSearchAgent
```

Our UCS agent finds the optimal solution in about 13 seconds, exploring over 16,000 nodes.

Make sure that your heuristic returns 0 at every goal state and never returns a negative value. Depending on how few nodes your heuristic expands, you'll get additional points:

Number of nodes expanded	Grade
more than 15000	4/16
at most 15000	8/16
at most 12000	12/16
at most 9000	16/16 (full credit; medium)
at most 7000	20/16 (optional extra credit; hard)

Table 1: Grading for question 7

Remember: **If your heuristic is inconsistent, you will receive no credit**, so be careful! Can you solve **mediumSearch** in a short time? If so, we're either very, very impressed, or your heuristic is inconsistent.

Question 8 (50 points): Suboptimal Search

Sometimes, even with A* and a good heuristic, finding the optimal path through all the dots is hard. In these cases, we'd still like to find a reasonably good path, quickly. In this section, you'll write an agent that always greedily eats the closest dot. **ClosestDotSearchAgent** is implemented for you in **searchAgents.py**, but it's missing a key function that finds a path to the closest dot.

Implement the function **findPathToClosestDot** in **searchAgents.py**. Our agent solves this maze (suboptimally!) in under a second with a path cost of 350:

```
python pacman.py -l bigSearch -p ClosestDotSearchAgent -z .5
```

Hint: The quickest way to complete **findPathToClosestDot** is to fill in the **AnyFoodSearchProblem**, which is missing its goal test. Then, solve that problem with an appropriate search function. The solution should be very short!

Your **ClosestDotSearchAgent** won't always find the shortest possible path through the maze. Make sure you understand why and try to come up with a small example where repeatedly going to the closest dot does not result in finding the shortest path for eating all the dots.

Submission

Once you are done, sign in to eLearning. You will be able to see **final project** under the course assignments section. Directly submit your **search.py** and **searchAgents.py** files. Please **do not** rename the files or upload the files in a zip file or folder (your homework will not be graded otherwise). **If you worked in a group, make sure to add your partner when you submit!**

Grading

We will be using an autograder as the main grading mechanism for this submission; the assignment files includes a copy (**autograder.py**) so we encourage you to use it before you submit! You may notice that the task description (and autograder) points sum up to a different number than the one indicated here; this is not an issue, we will be using an updated version that normalizes scoring for your final submission. The autograder is pretty sophisticated but you may still run into some very specific (rare) cases where it does not behave correctly. If you believe the autograder made a mistake when grading your homework, feel free to stop by your TA's office hours so you can discuss it with him. Any necessary corrections will be made to ensure you receive due credit for your work.

Important note: We will be using MOSS for plagiarism detection on all programming assignments (you can look it up, it's quite powerful). Any students caught cheating will receive a 0 on the corresponding assignment and will be reported according to university regulations.